

Санкт-Петербургский государственный университет

Бадмаев Ананда Аюрович

Выпускная квалификационная работа

***Разработка и реализация модуля ядра Linux для эффективного выделения
памяти под объекты малого размера***

Уровень образования: бакалавриат

Направление: 02.03.02 «Фундаментальная информатика и информационные
технологии

ООП: «Программирование и информационные технологии»

Научный руководитель:
профессор, кафедра компьютерного
моделирования и многопроцессорных систем,
д.т.н. Дегтярев Александр Борисович

Рецензент:
доцент, кафедра компьютерного
моделирования и многопроцессорных систем,
к.ф.-м.н. Корхов Владимир Владиславович

Санкт-Петербург
2022

СОДЕРЖАНИЕ

Введение.....	3
Постановка задачи.....	5
Обзор литературы.....	6
Глава 1. Особенности управления памятью в Linux.....	7
1.1 Пространство ядра и пространство пользователя.....	7
1.2 Виртуальная память.....	7
1.3 Kmalloc.....	8
1.4 Заготовленные кэши.....	8
1.4 Распределитель страниц.....	9
1.5 Вывод.....	9
Глава 2. Zpool API.....	10
2.1 Zbud.....	10
2.2 Z3fold.....	12
2.3 Zsmalloc.....	13
2.4 Вывод.....	14
Глава 3. Ztree.....	15
3.1 Идеи для повышения производительности аллокаторов.....	15
3.2 Организация блоков и деревьев.....	15
3.3 Приемы оптимизации.....	16
3.4 Отображение идентификаторов.....	17
3.5 Вывод.....	17
Глава 4. Инструменты тестирования.....	18
4.1 Zswap.....	18
4.2 Zram.....	18
4.3 Qemu.....	19
4.4 Fio.....	20
4.5 Ktime.....	20
4.6 Вывод.....	21
Глава 5. Сравнительное тестирование.....	22
5.1 Среднее время zpool операций.....	22
5.2 Худшее время zpool операций.....	24
5.3 Частота превышения порогов zpool операциями.....	25
Выводы.....	27
Заключение.....	28
Список литературы.....	29
Приложения.....	31

Введение

Распространение вычислительной техники в различных сферах человеческой деятельности сопровождается развитием операционных систем. Разнообразие вычислительных устройств порождает множество типов ОС: мобильные, распределенные, реального времени, для суперкомпьютеров, для встроенных систем и другие. Стоит отметить, что это деление весьма условно — одна и та же ОС может быть отнесена к нескольким разным типам, между которыми отсутствуют четкие границы.

В общем случае современные ОС состоят из трех основных компонент: ядро, системные библиотеки, оболочка и прикладные программы. Ядро является центральной частью операционной системы, управляет ее ресурсами и процессами. Многие, на первый взгляд, совершенно различные операционные системы, такие как Android для смартфонов и Red Hat Enterprise Linux, используемый в основном на серверах, работают на одном общем ядре, пусть и с разными конфигурациями и наборами дополнительных патчей.

На данный момент операционные системы на базе ядра Linux используются повсеместно: домашние компьютеры, серверы, мобильные устройства, рабочие станции, суперкомпьютеры. В последнее десятилетие бурно развиваются дистрибутивы Linux для применения во встраиваемых системах. Новые версии ядра поддерживают широкий спектр архитектур — не только классические x86 и ARM, но и более экзотические, такие как RISC-V.

Вычислительные устройства могут в процессе работы сталкиваться с нехваткой оперативной памяти и простое увеличение ее объема не всегда возможно по техническим или экономическим причинам. Особенно это актуально для встраиваемых и мобильных систем, а также недорогих ноутбуков. Данную проблему можно решать и другим способом — сжатием неиспользуемых страниц памяти для экономии места в ОЗУ. В сжатом виде такие страницы занимают меньший объем и поэтому могут более компактно размещаться в некотором едином хранилище.

В зависимости от алгоритма сжатия и содержимого размер сжатой страницы может варьироваться в широких пределах, то есть в одних случаях он может почти не измениться по сравнению с размером исходной страницы, а в других — сократиться в несколько раз. С учетом того, что объекты со временем добавляются в хранилище и удаляются из него, возникает проблема фрагментации, которая способна свести на нет экономию памяти за счет сжатия. Другим крайне важным аспектом является производительность, так как при слишком низкой скорости операций со страницами из хранилища, может оказаться, что использование простой подкачки страниц будет эффективнее чем их сжатие. Для систем реального времени также стоит учитывать задержки, связанные со временем выполнения операций в худшем случае.

В данной работе будет представлен новый аллокатор, названный `ztree`. Главной целью при его разработке было достичь сочетания высокой плотности хранения и скорости работы, а также низких задержек.

Постановка задачи

Целью данной работы является разработка и реализация эффективного модуля ядра Linux для выделения памяти для сжатых страниц. Для достижения поставленной цели требуется решить несколько задач, которые приведены ниже.

Задачи:

- Изучить основы управления памятью в Linux
- Рассмотреть существующие решения и изучить их исходный код
- Предложить идеи для повышения эффективности аллокаторов
- Реализовать данные идеи и провести отладку кода
- Выбрать подходящие инструменты тестирования
- Провести с их помощью сравнительное тестирование реализованного модуля и существующих решений

Обзор литературы

Общая информация об ОС Linux, в том числе и об устройстве ядра, может быть найдена в [1] и [3]. Ознакомиться с документацией по подсистеме управления памятью можно в [2]. В [4] подробно описывается разработка модулей и драйверов для ядра Linux.

По ссылкам [5], [6], [7], [8] доступна информация об общем интерфейсе для хранения сжатых страниц и совместимых с ним аллокаторах. В [9] и [10] представлена документация по модулям ядра, которые используют рассматриваемые аллокаторы для своей работы. Информация об инструментах тестирования и отладки может быть найдена в [11], [12], [13].

Глава 1. Особенности управления памятью в Linux

1.1 Пространство ядра и пространство пользователя

Linux разделяет всю доступную системе основную память между ядром и пространством пользователя. Пространство ядра — защищенная область памяти, в которой хранится и выполняется код самого ядра и его расширений, процессор в режиме ядра имеет полный доступ ко всем инструкциям. В пространстве пользователя выполняются пользовательские процессы, у которых доступ к ресурсам системы ограничен — для многих действий, таких как выделение памяти и доступ к аппаратному обеспечению, требуется обращение к ядру через системные вызовы [1, с. 826].

1.2 Виртуальная память

В Linux физическая память поделена на страницы, размер которых зависит от архитектуры процессора, но в большинстве случаев он равняется 4 КБ. При этом каждая страница физической памяти может отображаться как одна или несколько виртуальных страниц при помощи таблицы страниц. При обращении к памяти процессор транслирует виртуальный адрес в физический. Старшие биты виртуального адреса используются для нахождения соответствующей страницы в таблице, а младшие — непосредственно соответствуют смещению относительно начала физической страницы [2].

Процессы могут использовать больше виртуальной памяти, чем имеется ОЗУ в системе с помощью подкачки страниц. При обращении процесса к отсутствующей в памяти странице происходит ошибка страницы. Она приводит к выгрузке на диск некоторой другой страницы, которая с наименьшей вероятностью будет использоваться в ближайшем будущем и загрузке в память запрошенной [3, с. 325].

1.3 Kmalloc

Базовым инструментом для выделения памяти в ядре является *kmalloc*. Это быстрая функция, которая выделяет физически непрерывную область памяти заданного размера. Содержимое полученной области не очищается. Освобождение выделенной памяти происходит с помощью *kfree*.

Хотя в целом работа с *kmalloc* и *kfree* похожа на работу с *malloc* и *free* из стандартной библиотеки языка Си, с функциями ядра есть дополнительные сложности, которые стоит учитывать. Кроме размера *kmalloc* принимает на вход еще один аргумент — целое число, в котором закодированы флаги выделения памяти. Существует большое число флагов, из которых для нас будут полезны *GFP_ATOMIC* и *GFP_KERNEL*. Флаг *GFP_ATOMIC* указывает, что процесс, запросивший память, не должен заснуть, то есть с данным флагом *kmalloc* может вызываться в атомарном контексте. *GFP_KERNEL* — наиболее часто используемый флаг, при его использовании процесс может заснуть в ожидании свободной страницы.

Kmalloc может выделять только области памяти, являющиеся степенями двойки. Поэтому при запросе произвольного числа байт происходит округление до степени двойки в большую сторону. Также стоит учитывать, что эта функция не может оперировать блоками памяти меньше чем 32 или 64 байт в зависимости от платформы и больше чем 128 Кбайт. Таким образом, *kmalloc* плохо подходит для выделения памяти под слишком малые или слишком большие объекты [4, с. 213].

1.4 Заготовленные кэши

Другим механизмом выделения памяти является распределитель фрагментов (*slab allocator*), который используется для частого выделения памяти под объекты одного и того же размера. Ядро может создавать специальные пулы памяти для подобных объектов. Эти пулы называются

заготовленными кэшами, их можно создавать с помощью функции *kmem_cache_create*. Эта функция принимает на вход несколько аргументов, среди которых важнейшим является *size* — размер объектов в заготовленном кэше. Выделить память для объекта можно с помощью *kmem_cache_alloc*, а освободить — с помощью *kmem_cache_free* [4, с. 217].

1.4 Распределитель страниц

Для выделения больших блоков памяти лучше запрашивать память в страницах напрямую через распределитель страниц (page allocator), который использует приятельский (buddy) алгоритм. Распределитель страниц может выделять только количество страниц памяти, являющееся степенью двойки, так например, запрос на выделение 5 страниц в случае успеха приведет к выделению блока из 8 последовательных страниц. Для получения памяти с помощью этого механизма могут использоваться функции *__get_free_page*, *__get_zeroed_page*, *__get_free_pages*. Данные функции и в других аспектах имеют похожее на *kmalloc* — поведение принимают на вход те же флаги GFP и выделяют физически непрерывные области памяти. Единственным существенным отличием является то, что они оперируют страницами, поэтому совершенно не подходят для выделения небольших блоков памяти [4, с. 221].

1.5 Вывод

В ядре Linux существует множество способов выделения памяти, каждый из которых имеет свои особенности. *Kmalloc* довольно универсален, но во многих случаях следует отдать предпочтение более специализированным способам выделения памяти.

Глава 2. Zpool API

В ядре Linux существует единый интерфейс для пулов хранения памяти — zpool API. Обычно в качестве хранимых объектов выступают сжатые данные. В общем случае интерфейс при действиях с объектами оперирует не указателями на них, а специальными идентификаторами, имеющими тип *unsigned long*. С помощью такого идентификатора можно получить указатель на объект, используя соответствующий метод. Подобный косвенный доступ к данным обусловлен тем, что потенциально объект может перемещен внутри хранилища и указатель на него станет недействительным [5].

Zpool API предоставляет множество методов как для работы с самим пулом, так и для управления объектами из него. Ниже перечислены самые важные из них.

- *zpool_create_pool* — создает пул с заданными параметрами
- *zpool_destroy_pool* — удаляет пул
- *zpool_malloc* — выделяет из пула память под объект заданного размера
- *zpool_free* — освобождает выделенную память
- *zpool_map_handle* — преобразует идентификатор в указатель на объект
- *zpool_unmap_handle* — вспомогательный метод, который вызывается после *zpool_map_handle* и завершения действий с объектом

Zpool API является лишь интерфейсом, реализация которого возложена на специальные бэкенды (аллокаторы). На данный момент ядро Linux поддерживает 3 стандартных бэкенда для сжатых страниц: *zbud*, *z3fold*, *zsmalloc*.

2.1 Zbud

Zbud — простейший аллокатор, который размещает в свободную страницу памяти 1 или 2 сжатых страницы. Таким образом *zbud* теоретически

обеспечивает компрессию до 2х, но на практике это значение может быть значительно ниже.

Zbud размещает первый объект в начало страницы, а второй — в конец. Преимущество такого подхода в том, что при освобождении одного из слотов зазор между ними сливается с освобожденным пространством, что приводит к максимально возможному количеству свободного пространства для нового объекта. Таким образом даже в худшем случае данный аллокатор не будет использовать больше памяти для хранения сжатых страниц, чем их размер в распакованном виде [6].

Zbud разбивает страницу на целое число кусков (в случае типовой страницы размером 4 Кбайт на 64 куска по 64 байт) и при действиях с объектами оперирует их размерами в кусках вместо точных значений. Данный бэкенд создает массив связанных списков — *unbuddied*, в которых хранится только один объект, то есть свободен второй слот. Размерность *unbuddied* совпадает с числом кусков при разбиении страницы. Каждый список из массива содержит только объекты одного и того же размера в кусках, то есть имеет фиксированное количество свободного пространства в страницах. При добавлении объекта в *unbuddied* ищется подходящий связанный список и объект помещается в первую страницу этого списка, из которого затем удаляется эта страница, так как оба слота в ней уже заняты [6].

В качестве идентификатора объекта выступает указатель на него, приведенный к `unsigned long`. Поэтому не требуется сложная процедура отображения идентификатора в указатель.

Zbud активно использует связанные списки для хранения данных:

- *unbuddied* — массив списков страниц
- *buddied* — список страниц, в которых оба слота заняты.
- *lru* — список, отслеживающий страницы в порядке LRU (Least Recently Used)

К плюсам zbud можно отнести простую логику работы и достаточно высокую производительность. Очевидным и главным недостатком zbud является ограничение на максимальную плотность хранения данных — две сжатых страницы в одной свободной странице.

2.2 Z3fold

Z3fold — развитие идеи zbud, в отличие от которого z3fold может хранить до 3 сжатых страниц в свободной странице. Z3fold теоретически обеспечивает компрессию до 3х, но на практике это значение может также сильно деградировать [7].

Z3fold может хранить 1, 2 или 3 объекта в одной странице и похожим на zbud образом использует связные списки для хранения данных. Данный аллокатор также разбивает страницу на куски и оперирует при действиях с объектами их размером в кусках. Z3fold дополнительно использует заготовленные кэши для более эффективной работы со вспомогательными структурами и очереди задач (*workqueue*) — легковесные потоки ядра, обеспечивающие параллелизм по задачам.

Основные структуры z3fold:

- *unbuddied* — список страниц, в которых хранится 2 объекта и имеется 1 свободный слот
- *stale* — список страниц, отмеченных для удаления
- *lru* — список, отслеживающий страницы в порядке LRU (Least Recently Used)
- *compact_wq* — очередь задач для асинхронного уплотнения данных
- *release_wq* — очередь задач для безопасного освобождения страниц

Также как и zbud, даже в худшем случае z3fold не будет использовать больше памяти для хранения сжатых страниц, чем их размер в распакованном виде.

Идентификатор объекта не является числовым значением указателя как в zbud, поэтому *zpool_map_handle* расходует заметно больше процессорного времени чем в zbud.

Z3fold обеспечивает более высокую как максимальную, так и среднюю плотность хранения данных чем zbud, но достигается это за счет существенного усложнения логики работы и некоторого снижения производительности.

2.3 Zsmalloc

Zsmalloc — бэкенд, специально разработанный для zram. В отличие от zbud и z3fold, которые хранят целое число сжатых страниц в свободной странице, zsmalloc связывает непоследовательные свободные страницы в большую виртуальную страницу, что значительно повышает плотность упаковки данных, но усложняет логику работы и вызывает проблемы с фрагментацией [8].

Как и вышеупомянутые аллокаторы zsmalloc разбивает объекты на классы по их размерам (255 классов в случае 4 Кбайт страницы) и использует связанные списки для хранения данных. Данный аллокатор ведет отдельный связный список для каждого класса и дополнительно отслеживает заполненность виртуальных страниц, которая может иметь 4 значения: ZS_EMPTY, ZS_ALMOST_EMPTY, ZS_ALMOST_FULL, ZS_FULL. Zsmalloc также использует заготовленные кэши и очереди задач для повышения производительности.

Основные структуры zsmalloc:

- *zspage* — структура, описывающая виртуальную страницу
- *fullness_list* — массив связанных списков, отслеживающий заполненность виртуальных страниц, для каждого класса создается свой массив
- *size_class* — структура, описывающая класс
- *mapping_area* — отвечает за отображение идентификаторов на адреса

Главным достоинством zsmalloc является плотность хранения данных как средняя, так и максимальная. У данного аллокатора фактически отсутствует ограничение на максимальную плотность хранения, которая упирается в возможности алгоритма сжатия. В среднем zsmalloc также имеет наилучшие показатели.

Из недостатков zsmalloc можно выделить сложную логику работы, невысокую производительность во многих случаях и строгую зависимость от наличия MMU в системе — без этого устройства модуль нельзя собрать из-за использования виртуальных отображений памяти.

2.4 Вывод

Стандартные аллокаторы полагаются на связные списки для хранения данных. При большом количестве элементов проход по списку происходит довольно медленно. Zbud и z3fold дополнительно отслеживают страницы по LRU, что помогает сократить количество итераций по списку в среднем, но у zpool операций все еще остается линейная сложность в худшем случае. Для систем реального времени это может приводить к существенным задержкам.

Глава 3. Ztree

3.1 Идеи для повышения производительности аллокаторов

Были предложены две основных идеи для повышения производительности аллокаторов — размещение сжатых объектов в блоках из нескольких физически последовательных страниц и использование сбалансированных деревьев вместо связных списков для хранения данных.

Хранение объектов в страничных блоках вместо одиночных страниц позволяет выиграть за счет уменьшения количества выделений памяти. Например, вместо 8 выделений памяти для 8 отдельных страниц, можно сразу выделить память для 8-страничного блока. Также использование блоков позволяет более плотно хранить объекты подобно `zmalloc`, но для этого не требуется виртуальных отображений страниц и наличия MMU в системе.

Использование сбалансированных деревьев позволяет проводить операции доступа, добавления и удаления объектов из пула за $O(\log N)$, где N — общее количество объектов в дереве. Это значительно уменьшает время работы `zpool` операций в худшем случае, что приводит к большей отзывчивости системы в целом при использовании `zswap` или `zram`, о которых будет рассказано позже.

Аллокатор `ztree` использует обе эти идеи для устранения недостатков `zbud`, `z3fold` и `zmalloc`. Более подробно о реализации `ztree` рассказано далее.

3.2 Организация блоков и деревьев

Подобно вышеописанным аллокаторам `ztree` разбивает объекты на классы по их размерам. В зависимости от класса страничные блоки состоят из 1, 2, 4 или 8 физических страниц, необходимая для них память быстро выделяется с помощью `__get_free_pages`. Подобно `zmalloc` одна часть объекта может храниться в одной странице, вторая часть в другой, но при этом не снижается производительность из-за модификации таблицы страниц виртуальными

отображениями. Две страницы, содержащие разные части объекта, будут всегда соседними, так как страницы в блоке физически последовательны. Блоки имеют целое, фиксированное для каждого типа число слотов — от 8 до 16, а также все слоты в блоках одного типа имеют одинаковый размер. В совокупности эти моменты упрощают логику работы и существенно повышают производительность.

Всего имеется 16 классов блоков, каждому из которых соответствует свое красно-черное дерево. То есть блоки организованы соответственно в 16 независимых деревьев. Подобная организация хранения позволяет параллельно проводить действия с несколькими объектами разных классов.

3.3 Приемы оптимизации

В целях оптимизации были задействованы различные техники, благодаря которым удалось существенно повысить производительность и плотность хранения.

Последний блок и текущий блок — каждое дерево отслеживает последний добавленный в него блок, а также блок, в который идет добавление новых объектов. Данные блоки, работая вместе, предотвращают лишние проходы по дереву. Также использование текущего блока с коэффициентом заполнения способствует борьбе с фрагментацией.

Коэффициент заполнения блока — в каждый блок был добавлен целочисленный эвристический коэффициент, который отражает степень незаполненности блока. При освобождении слота в каждом блоке происходит сравнение коэффициентов из него и текущего блока, если значение в блоке превышает значение из текущего блока, то происходит переключение — этот блок становится текущим блоком.

Заготовленные кэши — память для метаданных блока выделяется с помощью `ktet_cache_alloc`. Учитывая частое выделение и освобождение памяти под эти объекты, это повышает производительность.

Иерархия блокировок — вводятся отдельные спин-блокировки для блоков и деревьев. Соответственно, операции затрагивающие все дерево, защищаются спин-блокировкой для дерева, а операции, затрагивающие только один блок — спин-блокировкой для блока. Подобная гранулярность блокировок позволяет повысить производительность на SMP системах, к которым относится большинство современных вычислительных устройств.

3.4 Отображение идентификаторов

В идентификаторе объекта с помощью битовых операций закодированы его класс, номер блока, номер слота. При отображении идентификатора сначала определяется класс объекта, который соответствует некоторому дереву. Затем ведется поиск по дереву с использованием номера блока в качестве ключа. В найденном блоке с помощью номера слота вычисляется адрес объекта.

Декодирование идентификатора происходит с помощью битовых операций. Вычисление адреса объекта осуществляется с использованием арифметики указателей. Таким образом, основную часть процессорного времени при отображении занимает поиск по дереву.

3.5 Вывод

Ztree потенциально предлагает плотность хранения данных, близкую к zsmalloc и высокую производительность. Также благодаря использованию красно-черных деревьев ожидается, что ztree будет иметь наилучшие показатели задержек zpool операций, что важно для задач реального времени.

Глава 4. Инструменты тестирования

4.1 Zswap

Zpool API главным образом используется в zswap — модуле ядра Linux, целью которого является уменьшение операций ввода-вывода с диском из-за подкачки страниц. Zswap перехватывает страницы, находящиеся в процессе выгрузки на диск, сжимает и пытается загрузить их в динамического размера хранилище, которое размещается в оперативной памяти. В случае нехватки свободного места в хранилище zswap вытесняет старые сжатые страницы в порядке LRU или перестает принимать новые.

Потенциальные сценарии использования:

- повышение производительности систем с ограниченным объемом ОЗУ
- увеличение продолжительности срока службы SSD и flash-накопителей из-за уменьшения количества дисковых операций

Модуль предоставляет возможность задать лимит потребления памяти хранилищем в процентах от общего объема, обнаружение страниц с одинаковым содержимым и так далее. Кроме того, zswap может использовать различные алгоритмы сжатия, что позволяет выбирать между скоростью и степенью сжатия [9].

С помощью данного модуля проводилась отладка ztree в процессе разработки, для этого применялись различные нагрузочные тесты памяти. Но в дальнейшем выяснилось, что zswap плохо подходит для сравнительного тестирования производительности, так как при его использовании трудно получить легко интерпретируемые численные результаты.

4.2 Zram

Zram — модуль ядра Linux, который позволяет создавать в оперативной памяти виртуальные блочные устройства. Страницы, записанные в подобное

устройство — zram диск, сжимаются и хранятся в самом диске, который имеет фиксированный размер. Zram диски имеют быстрый ввод-вывод, а сжатие позволяет экономить оперативную память. Такой диск может использоваться не только для подкачки страниц, но и в качестве временного хранилища или даже диска общего назначения. В модуле предусмотрено множество настроек, такие как количество дисков для создания, их размер, используемый алгоритм сжатия и другие [10].

Zram намного лучше подходит для измерения производительности аллокаторов, так как позволяет для этого использовать стандартные утилиты для тестирования дисков, с помощью которых можно получить конкретные численные значения тех или иных параметров. Хотя данный модуль более универсален в применении чем zswap, он не использует zpool API, из-за чего строго привязан к одному аллокатору — zsmalloc. Для решения данной проблемы на новую версию ядра был портирован патч, позволяющий zram использовать любой совместимый с zpool аллокатор.

4.3 Qemu

Qemu — универсальный эмулятор платформ с открытым исходным кодом. Обычно Qemu используется в режиме полной эмуляции (system emulation), в котором эмулируется целиком вся аппаратная платформа, то есть процессор, память, устройства ввода-вывода и так далее. Данном режим поддерживает как полную эмуляцию процессора, так и аппаратную виртуализацию, при которой код гостевой ОС напрямую исполняется на процессоре хоста [11].

Второй доступный режим — эмуляция пространства пользователя (user space emulation или user mod emulation). В нем Qemu может запускать программы, скомпилированные для одного процессора, на другом процессоре. Этот режим может использоваться только в ОС Linux и BSD, а также не поддерживает аппаратную виртуализацию [11].

Дополнительно Qemu предоставляет набор вспомогательных консольных утилит, позволяющих создавать и модифицировать образы дисков. Например, `qemu-img` дает возможность создавать образы дисков, на которые в дальнейшем можно устанавливать различные ОС.

4.4 Fio

Fio — инструмент для создания потоков или процессов, которые выполняют операции ввода-вывода некоторого типа, указанного пользователем. Данная утилита поддерживает широкий спектр платформ: Linux, Solaris, Windows, FreeBSD и другие. Fio принимает множество параметров, поэтому обычно для запуска используется отдельный файл с описанием задачи [12].

Основные параметры:

- I/O type — тип операций ввода-вывода, которые могут быть последовательными или случайными. Эти операции включают в себя чтение, запись и комбинацию чтения/записи.
- Block size — размер блока данных при операциях ввода-вывода. Можно указать единственное значение или диапазон.
- I/O size — общий объем данных для чтения и записи.
- Target file/device — тестируемое устройство или файл.

Данная утилита подходит для тестирования производительности блочных устройств. Она предоставляет возможность замерить скорость чтения и записи `zram` диска с конкретным аллокатором.

4.5 Ktime

Результаты первых тестов с помощью `fio` показали не слишком большую разницу в производительности между `zram` дисками с разными бэкендами за исключением `z3fold`, с которым диск выделялся среди остальных низкой

скоростью записи. Проблема в том, что скорость дисковых операций `zram` зависит еще и от алгоритма сжатия. Другими словами время операций чтения и записи включает в себя время на распаковку и сжатие страницы соответственно. Даже при использовании наиболее быстрого алгоритма сжатия `lz4` разница в скорости между бэкендами размывается. Для получения более точных значений производительности конкретных аллокаторов необходимо измерять непосредственно время операций `zpool API`.

Было решено измерять время основных `zpool` операций: `zpool_malloc`, `zpool_free`, `zpool_map`, `zpool_unmap`. Для этого в `zpool API` были добавлены счетчики и и замеры интервалов времени с помощью `ktime`. Функции из `ktime` объявлены `linux/timekeeping.h` и позволяют определять текущий момент времени. Из нескольких различных вариантов функций для измерения времени было решено использовать `ktime_get_ns`. Эта функция предназначена для измерения коротких интервалов времени с высокой точностью, она возвращает текущее относительное время в наносекундах [13].

4.6 Вывод

На практике скорость работы `zram` с различными аллокаторами не будет отличаться слишком сильно, так как основной вклад в среднее время дисковых операций `zram` вносит скорость алгоритма сжатия. Но при этом время операций аллокатора в худшем случае может потенциально приводить к заметным задержкам в работе системы.

Глава 5. Сравнительное тестирование

Тестирование производительности аллокаторов проводилось при помощи утилиты `fiio` и `zram` дисков в эмуляторе `qemu`. Для этого использовалась виртуальная платформа ARM64, на которой был установлен дистрибутив Debian 11 с модифицированным ядром Linux.

Для тестирования использовались `zram` диски размером 256 и 768 Мбайт, а также с помощью аргумента утилиты `fiio` было задано два уровня сжатия для блоков данных при операциях ввода-вывода. Комбинируя размер диска и уровень сжатия, можно получить 4 тестовых сценариях.

5.1 Среднее время `zpool` операций

Среднее время `zpool` операций является достоверным и практичным показателем, устойчивым к отдельным выбросам. В таблицах 5.1.1 — 5.1.4 приведены результаты тестов в 4 сценариях.

Таблица 5.1.1 – Сравнение среднего времени операций в мкс при размере `zram` диска 256 Мбайт и низком уровне сжатия

Операция	Zbud	Z3fold	Zsmalloc	Ztree
malloc	9.0	24.6	8.3	3.1
free	4.7	32.6	4.5	2.8
map	0.2	2.8	2.8	3.0
unmap	0.2	1.0	1.2	0.9

Таблица 5.1.2 – Сравнение среднего времени операций в мкс при размере `zram` диска 768 Мбайт и низком уровне сжатия

Операция	Zbud	Z3fold	Zsmalloc	Ztree
malloc	9.1	23.3	9.1	3.5
free	3.9	20.7	4.8	2.3
map	0.2	2.7	3.0	3.1
unmap	0.2	1.0	1.2	0.9

Таблица 5.1.3 – Сравнение среднего времени операций в мкс при размере zram диска 256 Мбайт и среднем уровне сжатия

Операция	Zbud	Z3fold	Zsmalloc	Ztree
malloc	3.6	22.7	8.9	3.0
free	1.4	32.0	4.7	2.9
map	0.2	2.9	2.3	2.9
unmap	0.2	1.0	0.8	0.9

Таблица 5.1.4 – Сравнение среднего времени операций в мкс при размере zram диска 768 Мбайт и среднем уровне сжатия

Операция	Zbud	Z3fold	Zsmalloc	Ztree
malloc	4.5	23.3	10.0	3.5
free	1.7	23.2	4.0	2.4
map	0.2	2.8	2.3	3.2
unmap	0.2	1.0	0.8	0.9

Резкое повышение средней скорости *malloc* и *free* у *zbud* при переходе от низкого уровня сжатия к среднему можно объяснить тем, что при низком сжатии размер объектов составляет более половины от размера страницы. Поэтому *zbud* вынужден для каждого объекта выделять новую страницу после обхода списка в попытке найти слот достаточного размера.

Средняя скорость *zpool* операций у всех аллокаторов существенно различается, тем не менее *zbud*, *ztree* и *zsmalloc* имеют приблизительно равную производительность дисковых операций *zram* даже при использовании быстрого алгоритма сжатия *lz4*. Только *z3fold* заметно выбивался в худшую сторону по показателям скорости дисковых операций из-за слишком медленных операций *malloc* и *free*, время которых сопоставимо со временем сжатия и распаковки страницы алгоритмом *lz4*.

5.2 Худшее время zpool операций

Для оценки эффективности аллокаторов в задачах реального времени имеет смысл вместо среднего время работы zpool операций рассматривать худшее время. В таблицах 5.2.1 — 5.2.4 приведены результаты тестов в 4 сценариях.

Таблица 5.2.1 – Сравнение худшего времени операций в мс при размере zram диска 256 Мбайт и низком уровне сжатия

Операция	Zbud	Z3fold	Zsmalloc	Ztree
malloc	0.4	1.2	1.0	0.4
free	5.3	7.1	1.2	0.3
map	0.5	0.3	0.4	0.3
unmap	0.3	0.3	1.2	0.2

Таблица 5.2.2 – Сравнение худшего времени операций в мс при размере zram диска 768 Мбайт и низком уровне сжатия

Операция	Zbud	Z3fold	Zsmalloc	Ztree
malloc	0.3	2.2	1.0	0.4
free	7.1	0.6	1.3	1.0
map	0.6	0.2	0.3	0.4
unmap	0.5	0.3	0.7	0.2

Таблица 5.2.3 – Сравнение худшего времени операций в мс при размере zram диска 256 Мбайт и среднем уровне сжатия

Операция	Zbud	Z3fold	Zsmalloc	Ztree
malloc	0.5	0.9	0.9	0.4
free	1.1	8.0	0.9	0.3
map	0.2	1.9	0.5	0.3
unmap	0.9	0.4	0.4	0.2

Таблица 5.2.4 – Сравнение худшего времени операций в мс при размере *zram* диска *zram* диска 768 Мбайт и среднем уровне сжатия

Операция	Zbud	Z3fold	Zsmalloc	Ztree
<i>malloc</i>	0.3	0.9	0.9	0.4
<i>free</i>	5.7	24.2	1.4	1.3
<i>map</i>	0.2	0.2	0.3	0.3
<i>unmap</i>	0.2	0.2	3.0	0.3

Zbud и *z3fold* показывают задержки базовых операций в несколько мс, что в некоторых случаях может быть неприемлемо. Эти значения сопоставимы со временем задержек жестких дисков.

Ztree имеет наименьшее время *zpool* операций в худшем случае. Время *malloc*, *map* и *unmap* не выходит за пределы 0.5 мс. В случае *free* иногда происходят превышения порога 1 мс, так как это самая трудоемкая операция в худшем случае. Потенциально *free*, кроме поиска по дереву, может приводить к удалению блока из дерева, если объект был единственным в данном блоке.

5.3 Частота превышения порогов *zpool* операциями

Показатели худшего времени работы чувствительны к внешним факторам, также полезно знать, насколько часто случаются выбросы. Для каждого аллокатора замерялась частота превышений определенных порогов *zpool* операциями. В таблицах 5.3.1 — 5.3.4 приведены результаты тестов в 4 сценариях.

Таблица 5.3.1 – Сравнение количество превышений порогов по времени при размере *zram* диска 256 Мбайт и низком уровне сжатия

Порог	Zbud	Z3fold	Zsmalloc	Ztree
0.5 мс	20	13	16	0
1 мс	18	6	6	0
2 мс	15	1	0	0
4 мс	2	1	0	0

Таблица 5.3.2 – Сравнение количество превышений порогов по времени при размере zram диска 768 Мбайт и низком уровне сжатия

Порог	Zbud	Z3fold	Zsmalloc	Ztree
0.5 мс	59	6	12	12
1 мс	58	4	5	1
2 мс	47	1	0	0
4 мс	11	0	0	0

Таблица 5.3.3 – Сравнение количество превышений порогов по времени при размере zram диска 256 Мбайт и среднем уровне сжатия

Порог	Zbud	Z3fold	Zsmalloc	Ztree
0.5 мс	11	49	6	1
1 мс	1	26	0	0
2 мс	0	14	0	0
4 мс	0	7	0	0

Таблица 5.3.4 – Сравнение количество превышений порогов по времени при размере zram диска 768 Мбайт и среднем уровне сжатия

Порог	Zbud	Z3fold	Zsmalloc	Ztree
0.5 мс	28	169	23	13
1 мс	26	144	14	3
2 мс	15	128	2	0
4 мс	4	109	0	0

При низком уровне сжатия zbud имеет большое число выбросов, в то время как z3fold показывает подобное поведение наоборот при среднем уровне сжатия. У zsmalloc умеренное число превышений порогов при всех сценариях. Ztree, как и ожидалось, демонстрирует наилучшие показатели в этом виде тестов.

Выводы

В ходе работы были получены следующие результаты:

- Изучены основы управления памятью в Linux
- Рассмотрены стандартные аллокаторы, проанализированы их достоинства и недостатки
- Предложены идеи для повышения производительности аллокаторов
- Разработан новый аллокатор ztree на основе предложенных идей
- Подобраны инструменты тестирования аллокаторов
- Проведено сравнительное тестирование ztree и существующих решений

Заключение

Были изучены специализированные способы выделения памяти в ядре Linux, такие как распределитель страниц и заготовленные кэши. Использование этих методов позволяет оптимизировать процесс выделения памяти.

Основным недостатком существующих в ядре аллокаторов является медленный обход связного списка при добавлении или удалении объектов. Также zbud и z3fold оперируют отдельными страницами, что существенно ограничивает плотность хранения объектов. В случае zsmalloc дополнительными недостатками выступают зависимость от ММУ и накладные расходы, связанные с модификацией таблицы страниц.

Благодаря использованию страничных блоков и красно-черных деревьев, представленный в данной работе аллокатор ztree предлагает плотность хранения данных, близкую к zsmalloc и высокую производительность.

Тесты показали, что стандартные аллокаторы могут иметь задержки при добавлении или удалении объектов, сопоставимые с задержками ввода-вывода в жестком диске. Все операции с объектами в ztree выполняются за $O(\log N)$, где N — общее число объектов в пуле. Это позволяет ограничить время выполнения операций в худшем случае по сравнению с существующими в ядре аллокаторами.

Список литературы

1 Э. Таненбаум, Х. Бос. Современные операционные системы. 4-е издание. — Спб.: Питер, 2015. — 1120 с.

2 Concepts Overview [Электронный ресурс]

// URL: <https://www.kernel.org/doc/html/latest/admin-guide/mm/concepts.html>

(дата обращения: 03.12.2021)

3 Р. Лав. Linux системное программирование. 2-е издание. — Спб.: Питер, 2014. — 448 с.

4 J. Corbet, A. Rubini, G. Kroah-Hartman. Linux Device Drivers. 3rd edition. — O'Reilly Media, 2005. — 616 с.

5 zpool.c [Электронный ресурс]

// URL: <https://github.com/torvalds/linux/blob/master/mm/zpool.c>

(дата обращения: 08.12.2021)

6 zbud.c [Электронный ресурс]

// URL: <https://github.com/torvalds/linux/blob/master/mm/zpool.c>

(дата обращения: 08.12.2021)

7 z3fold [Электронный ресурс]

// URL: <https://www.kernel.org/doc/html/latest/vm/z3fold.html>

(дата обращения: 08.12.2021)

8 zsmalloc [Электронный ресурс]

// URL: <https://www.kernel.org/doc/html/latest/vm/zsmalloc.html>

(дата обращения: 08.12.2021)

9 zswap [Электронный ресурс]

// URL: <https://www.kernel.org/doc/html/latest/admin-guide/mm/zswap.html>

(дата обращения: 05.12.2021)

10 zram: Compressed RAM based block devices [Электронный ресурс]

// URL: <https://www.kernel.org/doc/Documentation/blockdev/zram.txt>

(дата обращения: 05.12.2021)

11 About QEMU [Электронный ресурс]

// URL: <https://www.qemu.org/docs/master/about/index.html>

(дата обращения: 23.04.2022)

12 Welcome to FIO's documentation [Электронный ресурс]
// URL: <https://fio.readthedocs.io/en/latest/> (дата обращения: 23.03.2022)

13 ktime accessors [Электронный ресурс]
// URL: <https://www.kernel.org/doc/html/latest/core-api/timekeeping.html>
(дата обращения: 12.05.2022)

Приложения

Код модуля `ztree` доступен по ссылке (чистая ветка для отправки на ревью):
<https://github.com/badmaev055663/linux/tree/ztree>

Для использования модуля необходимо клонировать данную ветку, при конфигурации ядра с помощью *make menuconfig* в разделе *Memory Managment options* выбрать опцию «*Simple allocator for compressed pages*». Далее собрать ядро следуя стандартной инструкции. После загрузки ядра в настройках **zswap**, указать «*ztree*» в качестве бэкенда. В целях безопасности полученное модифицированное ядро лучше использовать в эмуляторе или виртуальной машине.

Рабочая ветка с тестовым кодом и вспомогательными патчами доступна по ссылке: <https://github.com/badmaev055663/linux/tree/ztree-testing>